

Capítulo 2

Procesos y Threads

2.1 Procesos

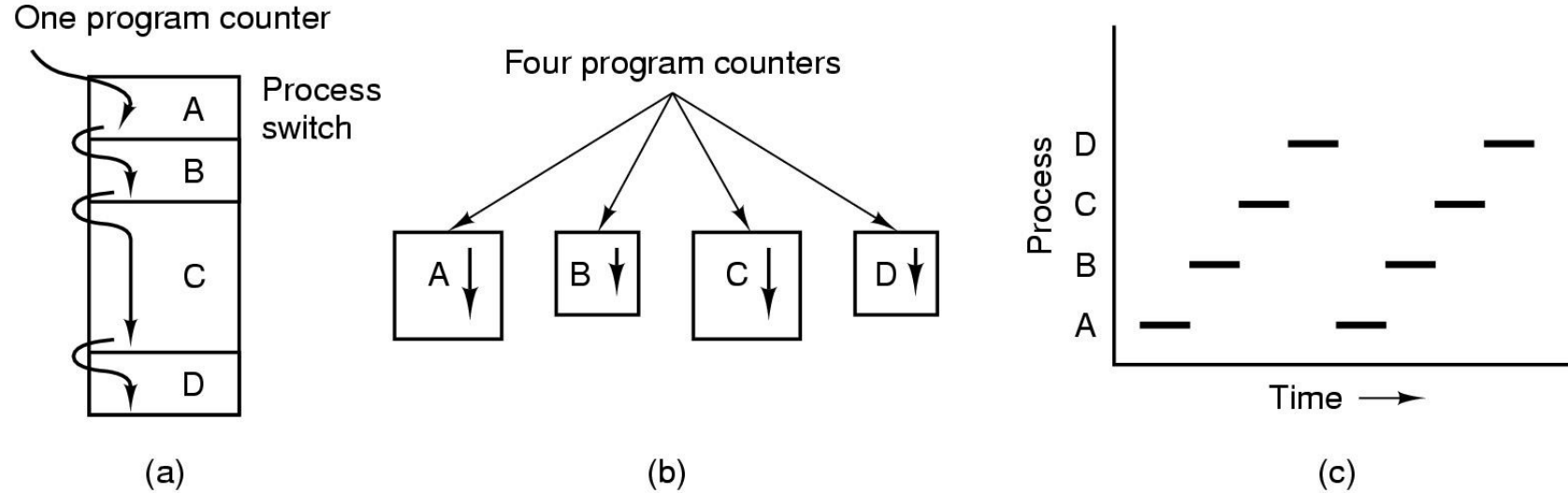
2.2 Threads

2.3 Comunicación entre procesos

2.5 Planificación

Procesos (1)

El Modelo de Proceso



- a. **Multiprogramación** de 4 programas
- b. Modelo conceptual de 4 procesos secuenciales independientes
- c. Solo un proceso activo en cada instante

Procesos (2)

Creación de procesos

Principales eventos que causan la creación de procesos

1. Inicialización del sistema (**demonios**) [comando ps]
2. Ejecución de la llamada al sistema para crear procesos
3. El usuario solicita un nuevo proceso (comando)
4. Inicio de un trabajo batch

Todos invocan `fork()`

Procesos (3)

Terminación de procesos

Condiciones que terminan un proceso

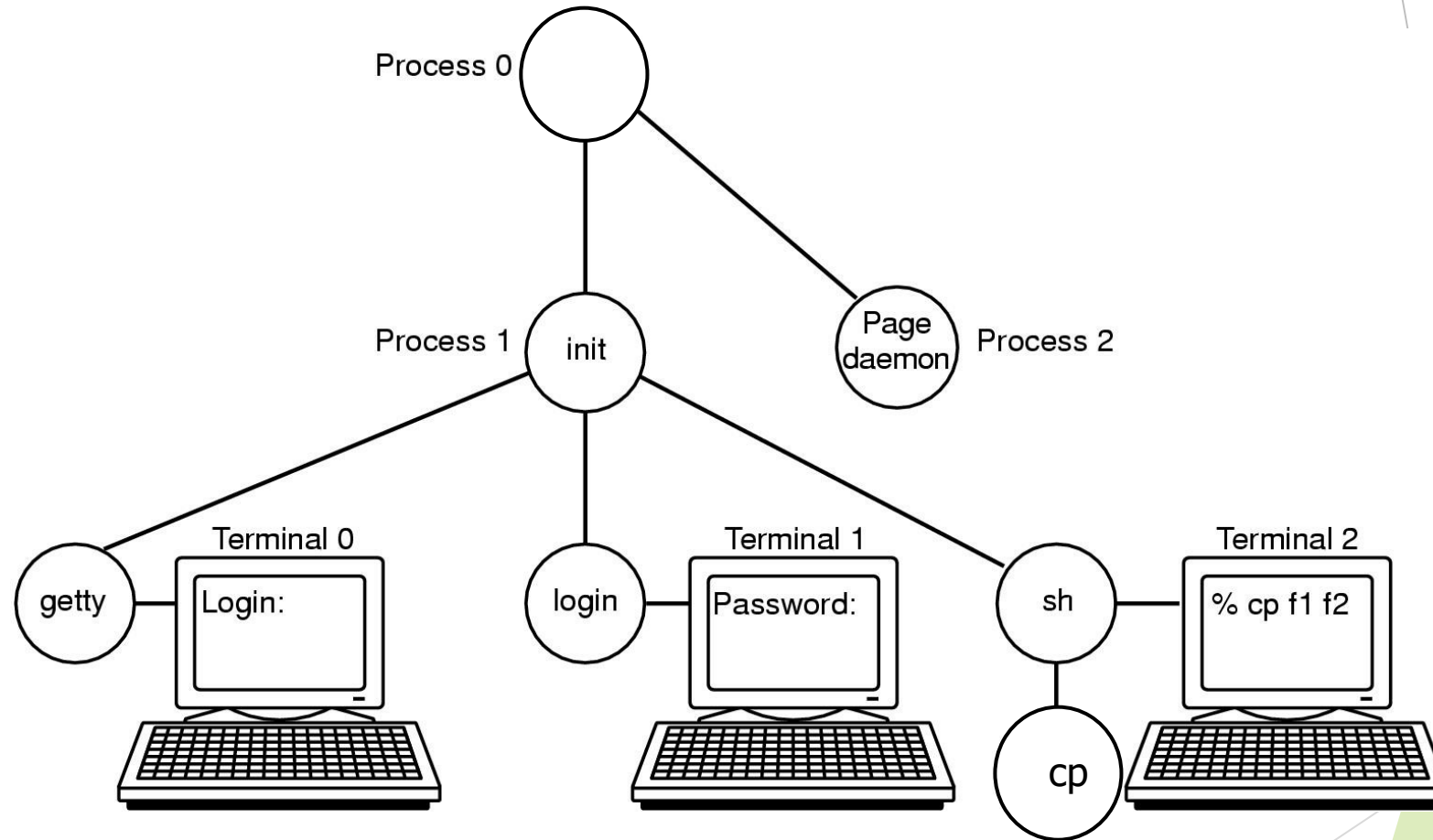
1. Normal exit (voluntario)
2. Error exit (voluntario)
3. Fatal error (involuntario)
4. Asesinado por otro proceso (involuntario, señal) o suicidado (voluntario, señal)

Procesos (4)

Grupos de procesos

- ▶ El proceso padre crea procesos hijos y los procesos hijos pueden crear sus propios procesos
- ▶ Jerarquía de procesos
 - ▶ UNIX llama a esto "**process group**"
- ▶ Windows no tiene el concepto de jerarquía de procesos
- ▶ Se puede operar con grupos de procesos (p. ejem.: enviar una señal.
- ▶ No confundir este grupo con el grupo de rwx rwx rwx

Booting UNIX



The sequences of processes used to boot some systems 6

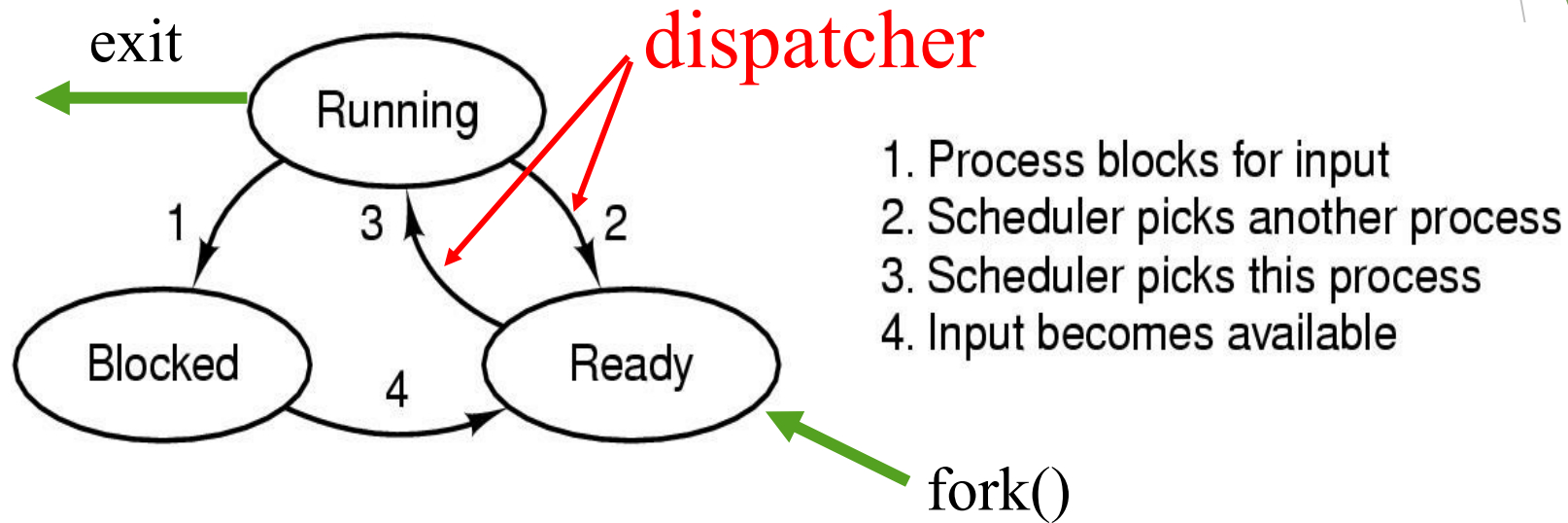
Procesos (5)

Grupos de procesos

- *init* lee /etc/ttys (número de terminales).
- *init* crea un proceso hijo por cada terminal.
- Los hijos ejecutan (exec) el programa *bin/login*.
- Esperan a que un usuario entre al sistema.
- *bin/login* ejecuta la shell correspondiente (en /etc/passwd)
- La shell crea un hijo por cada comando.

Procesos (6)

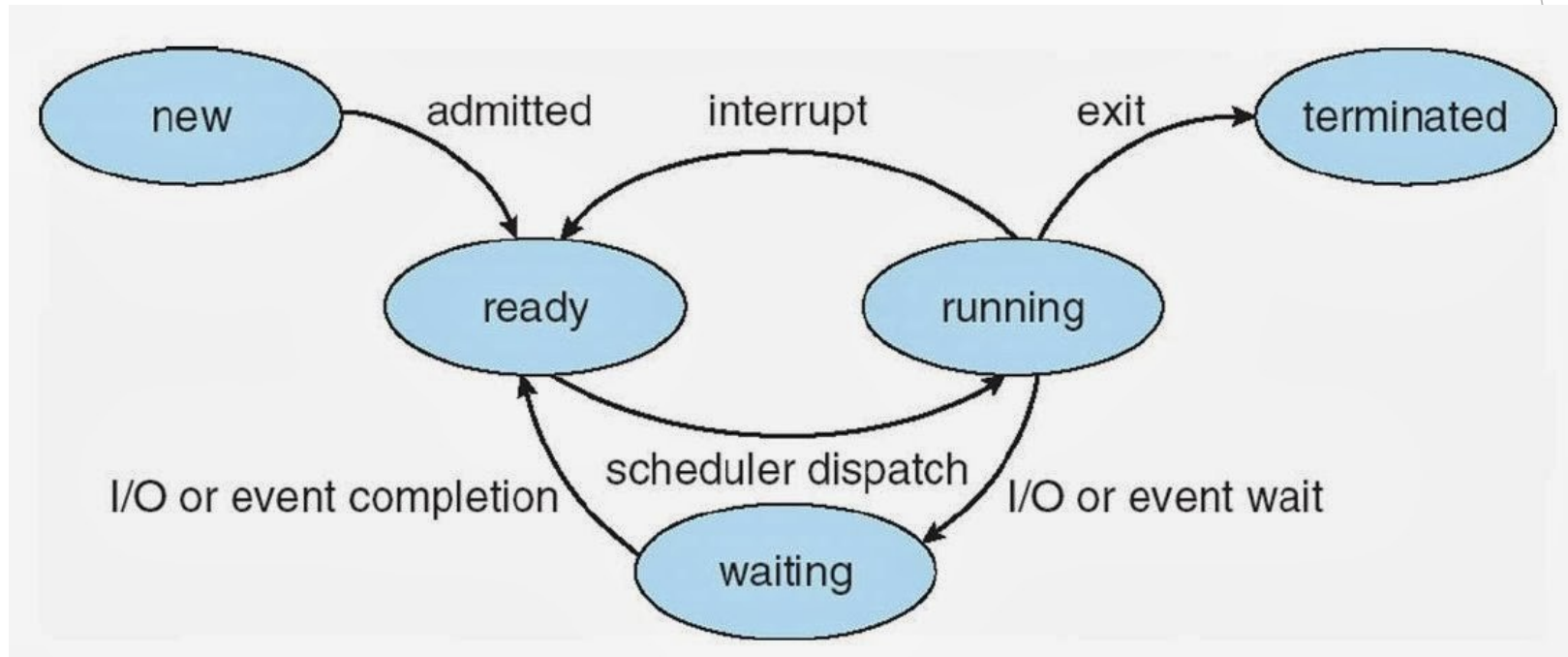
Estados de procesos



► Posibles estados de un procesos

- Running (ejecutándose)
- Blocked (bloqueado)
- Ready (preparado)

Modelo 5 estados



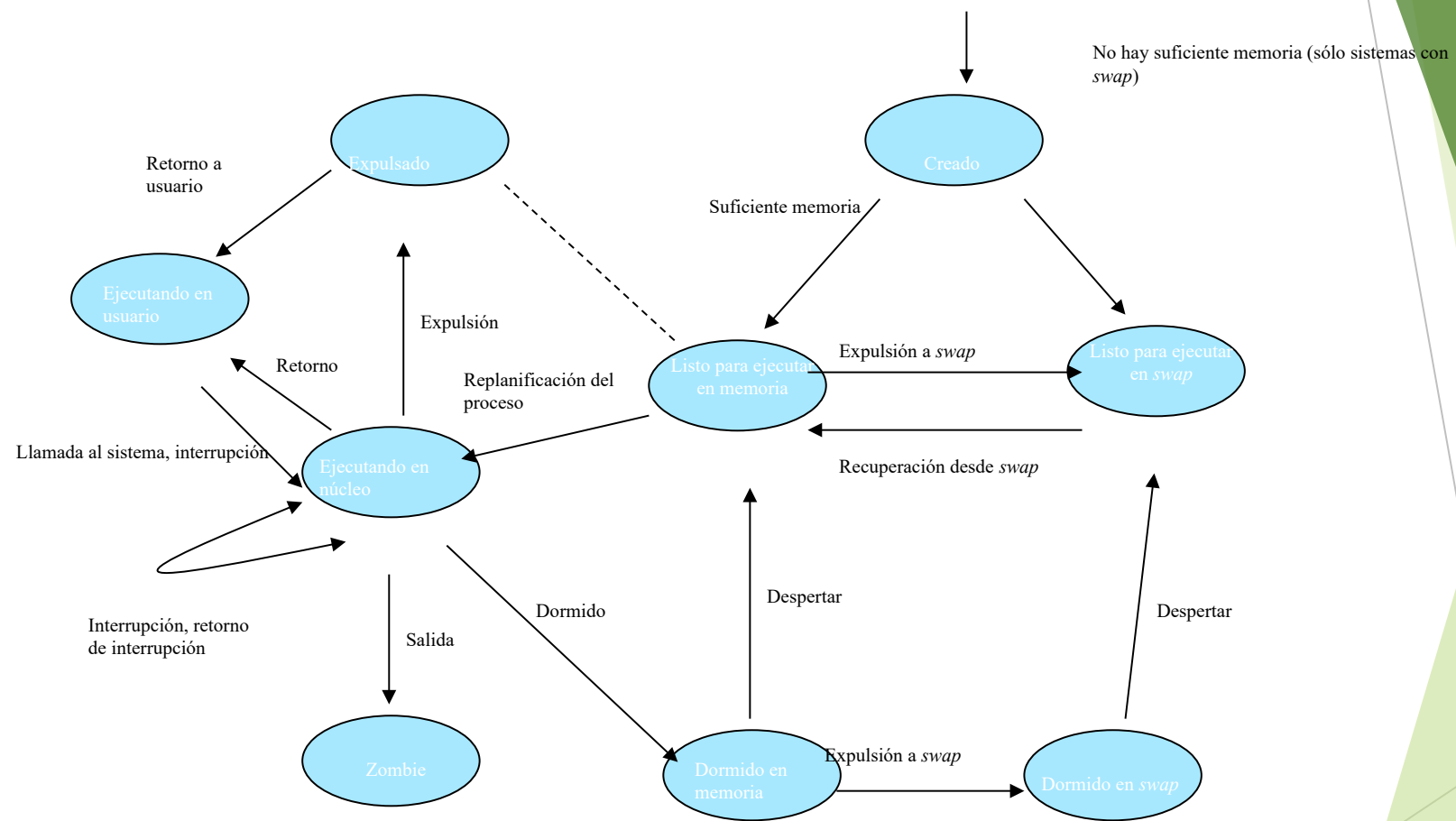
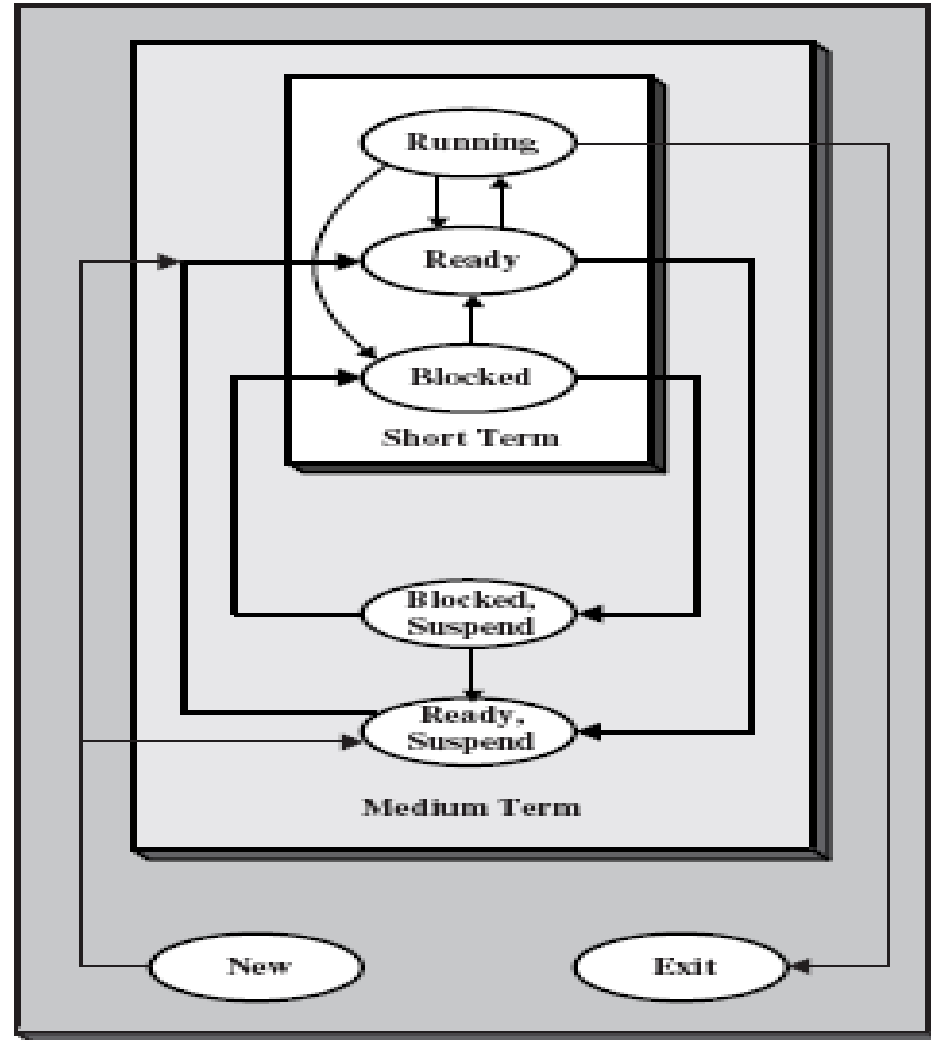


Diagrama de transiciones entre estados de procesos UNIX

Planificadores



Procesos (7)

Implementación de procesos

Process management	Memory management	File management
Registers Program counter Program status word Stack pointer Process state Priority Scheduling parameters Process ID Parent process Process group Signals Time when process started CPU time used Children's CPU time Time of next alarm	Pointer to text segment Pointer to data segment Pointer to stack segment	Root directory Working directory File descriptors User ID Group ID

Campos en una entrada a la tabla de procesos

Procesos (8)

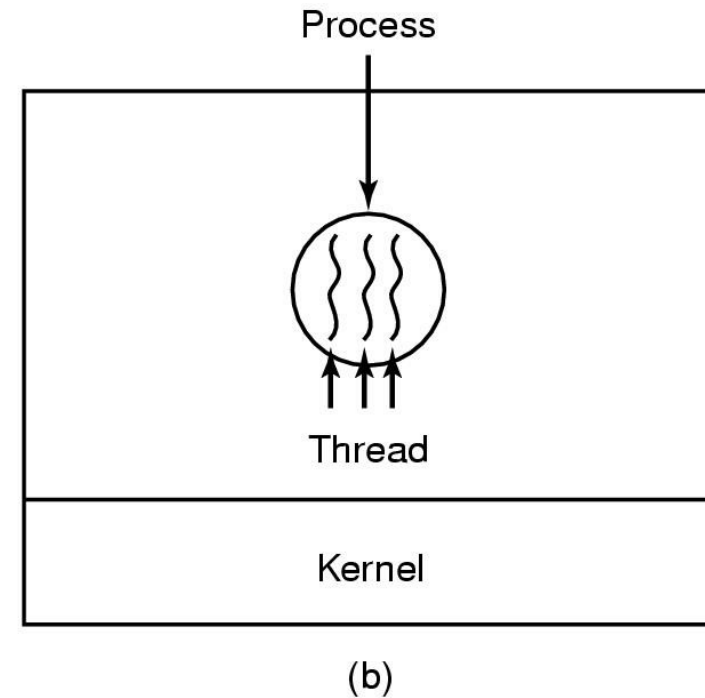
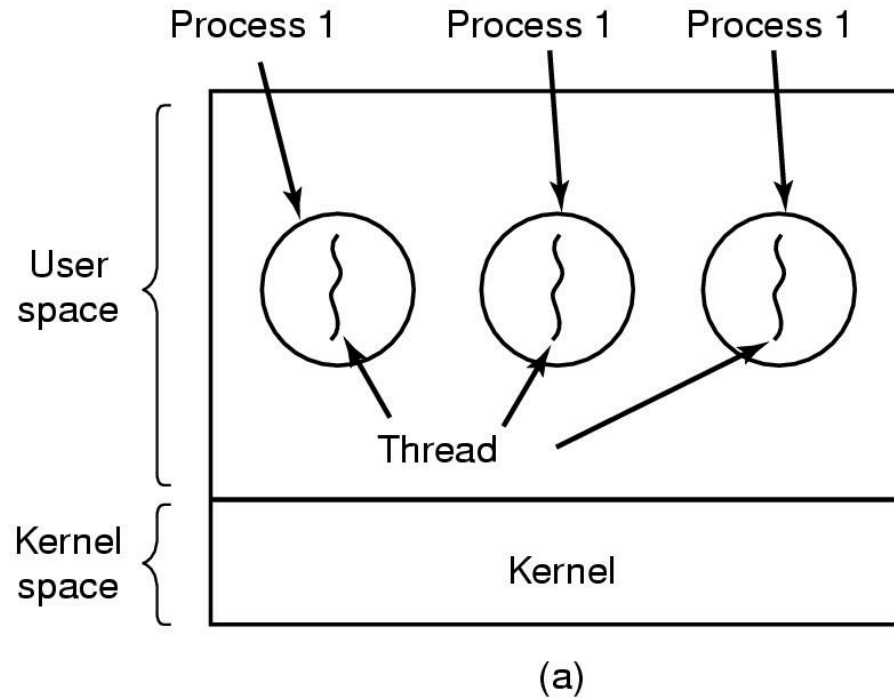
Implementación de procesos

1. Hardware stacks program counter, etc.
2. Hardware loads new program counter from interrupt vector.
3. Assembly language procedure saves registers.
4. Assembly language procedure sets up new stack.
5. C interrupt service runs (typically reads and buffers input).
6. Scheduler decides which process is to run next.
7. C procedure returns to the assembly code.
8. Assembly language procedure starts up new current process.

Esqueleto de lo que hace el SO cuando llega una interrupción.

Threads (1)

El modelo de thread



- (a) Tres procesos cada uno con un thread
- (b) Un proceso con tres threads

Threads (2)

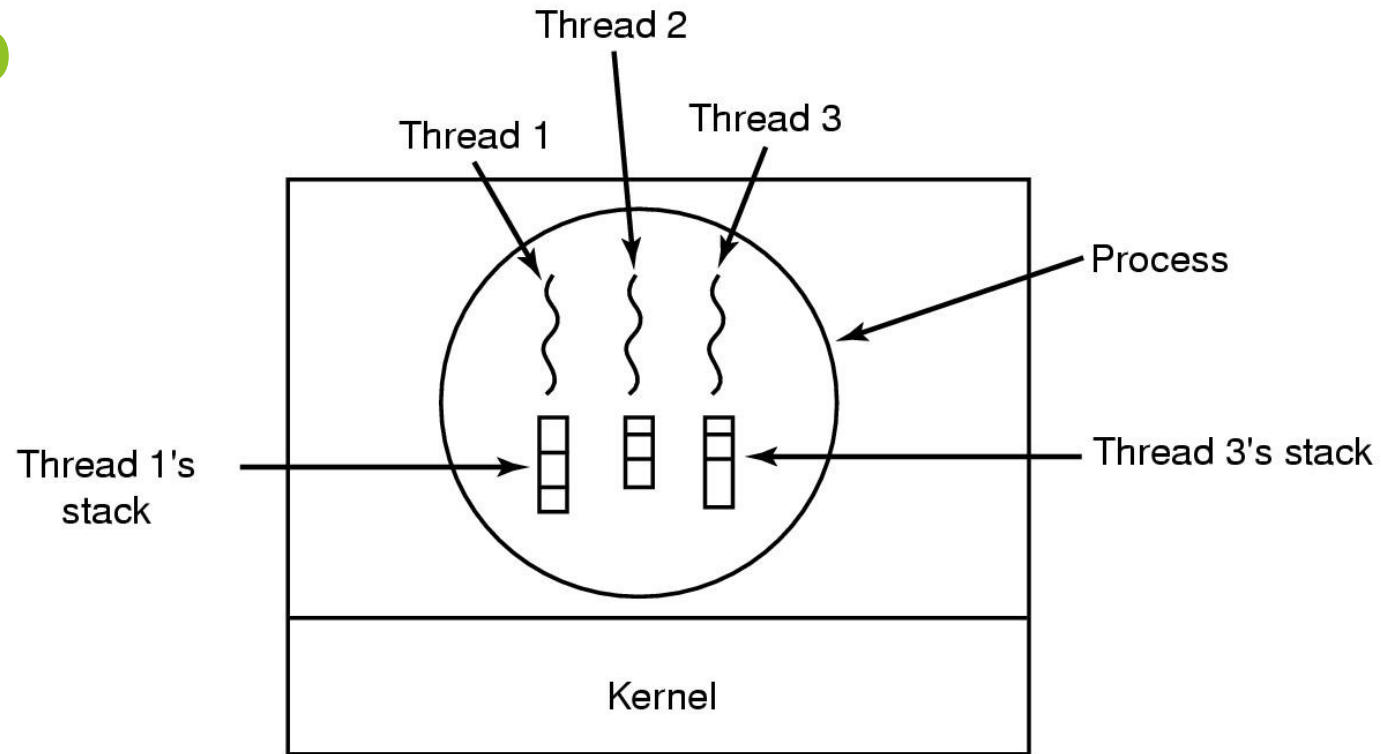
El modelo de thread

Per process items	Per thread items
Address space	Program counter
Global variables	Registers
Open files	Stack
Child processes	State
Pending alarms	
Signals and signal handlers	
Accounting information	

- Items compartidos por todos los threads en un proceso
- Items privados a cada thread

Threads (3)

El modelo



Cada thread tiene su propio stack

Threads (4)

El modelo de thread

Operaciones sobre threads

- Crear un thread

```
int pthread_create (pthread_t *identThread, const pthread_attr_t *atributos,  
                  void *(*funcion, void*), void *argumento)
```

```
pthread_create (&tA, NULL, escribir, &a)
```

siendo:

```
pthread_t tA;  
void *escribir (void *letra) {...}
```

- Esperar a que termine otro thread

```
int pthread_join (pthread_t thread, void **estado)
```

```
pthread_join (tA, NULL)
```

- Terminar un thread

```
void pthread_exit (void *estado)
```

```
pthread_exit (NULL)
```

También:

```
pthread_self()
```

```
pthread_yield()
```

Threads (5)

El modelo de thread

Problemas con threads

- Fork. ¿El hijo hereda los threads del padre?
NO: puede no funcionar
SI: un thread en el padre bloqueado por E/S
¿quién lee? ¿el thread del padre? ¿el del hijo?
¿los dos?
- Los threads comparten estructuras de datos. Si un thread cierra un fichero mientras otros están leyendo ¿qué pasa?

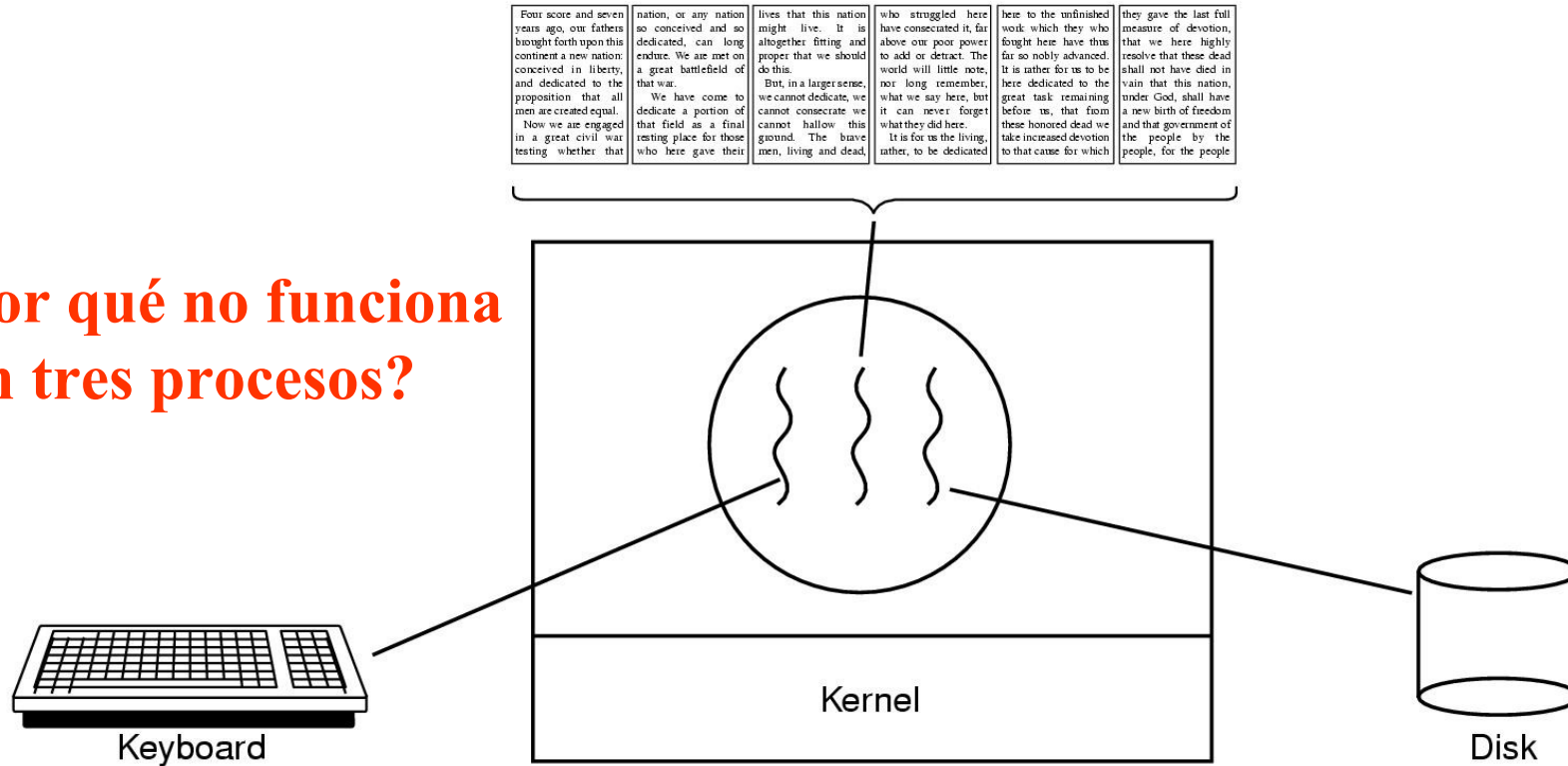
¿Para qué threads si hay procesos?

- Se simplifica el modelo de programación.
- Es más rápido crear/destruir threads que procesos.
- Razones de rendimiento.

Threads (6)

Uso de threads

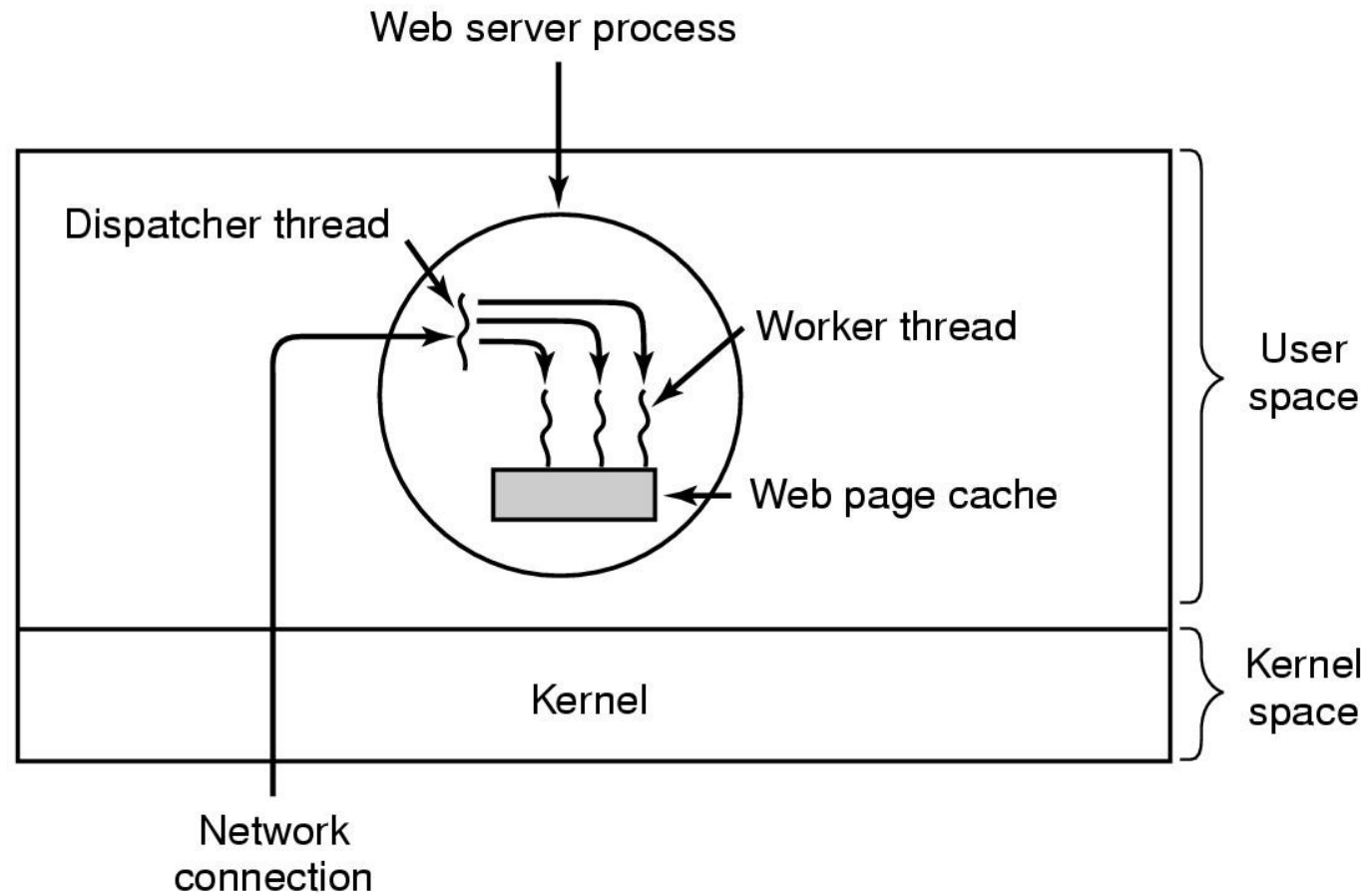
¿Por qué no funciona con tres procesos?



Un procesador de textos con 3 threads₁₉

Threads (7)

Uso de threads



Un servidor web multithread

Threads (8)

Uso de threads

```
while (TRUE) {  
    get_next_request(&buf);  
    handoff_work(&buf);  
}
```

(a)

```
while (TRUE) {  
    wait_for_work(&buf)  
    look_for_page_in_cache(&buf, &page);  
    if (page_not_in_cache(&page)  
        read_page_from_disk(&buf, &page);  
    return_page(&page);  
}
```

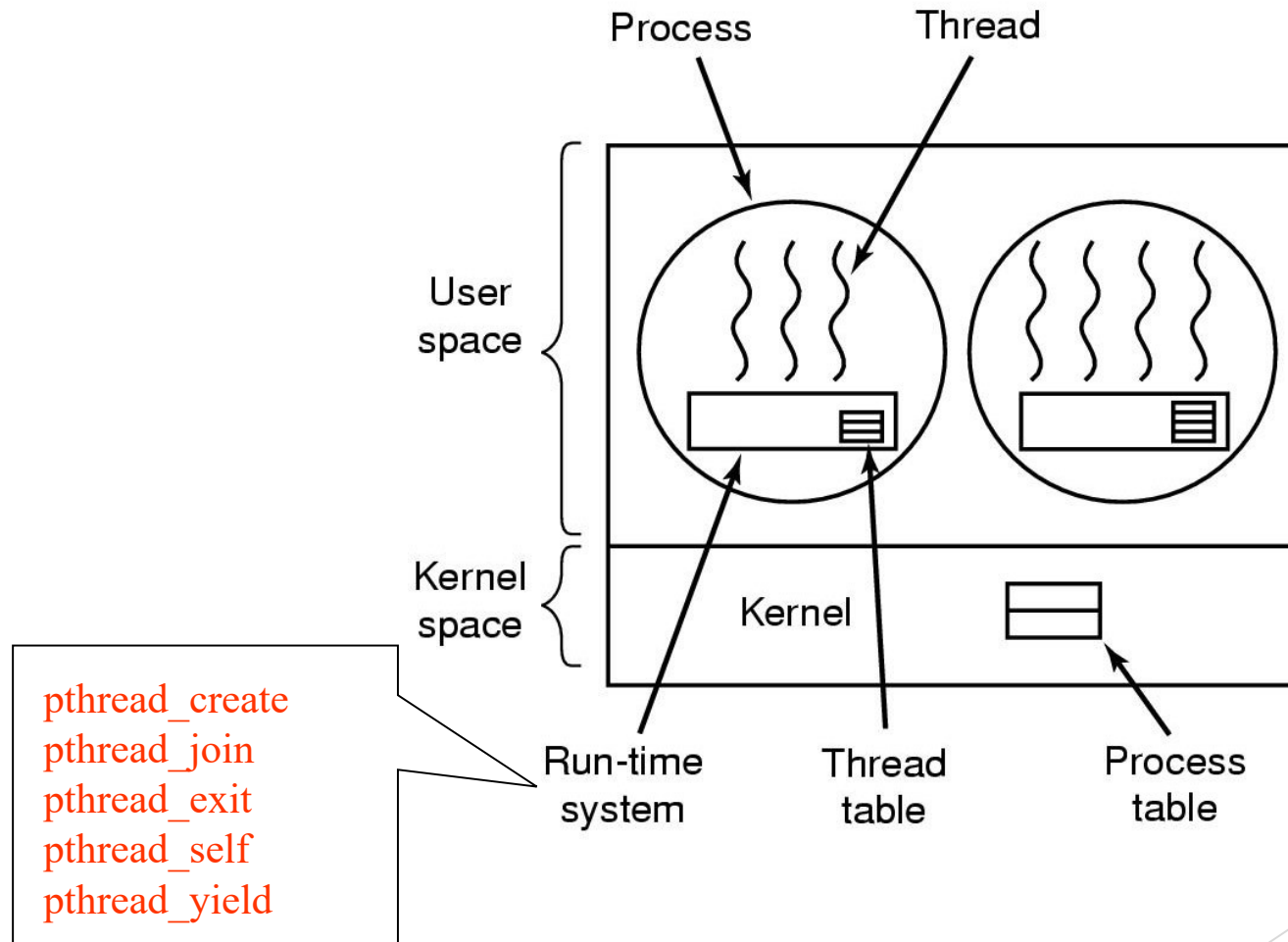
(b)

- Dispatcher (a)
- Worker (b)

¿Cómo sin threads?

Threads (9)

Implementación de threads en espacio de usuario



Threads (10)

Implementación de threads en espacio de usuario

Ventajas:

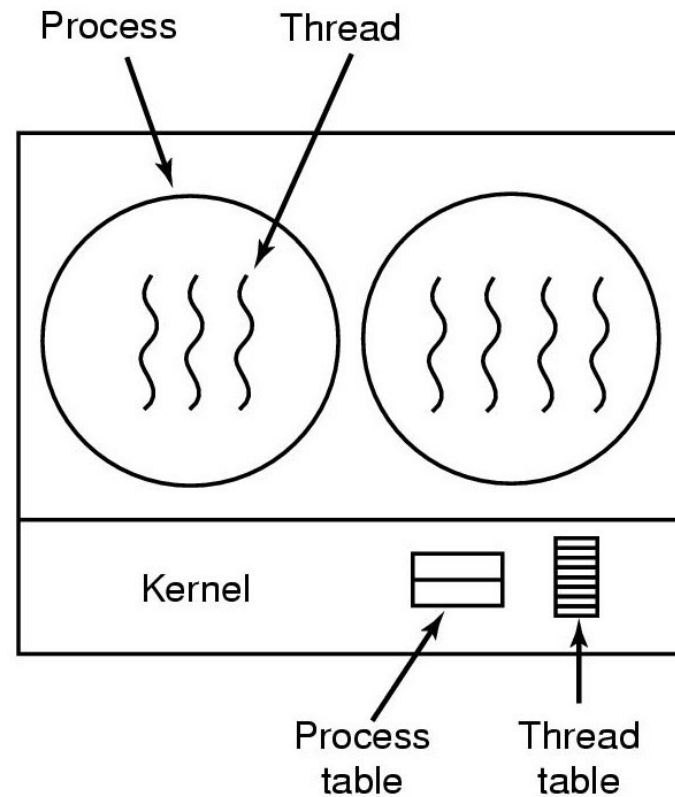
- Se puede tener threads en sistemas operativos que no los soportan.
- El switching es mucho más rápido que si hay que comunicarlo al kernel.
- Cada proceso puede tener su propio algoritmo de planificación threads.

Desventajas:

- ¿Cómo se implementan las llamadas al sistema bloqueantes?
- Un thread provoca una falta de página: bloquea al proceso
- los threads tienden a monopolizar la CPU

Threads (11)

Implementación de threads en espacio de kernel



Threads (12)

Implementación de threads en espacio de kernel

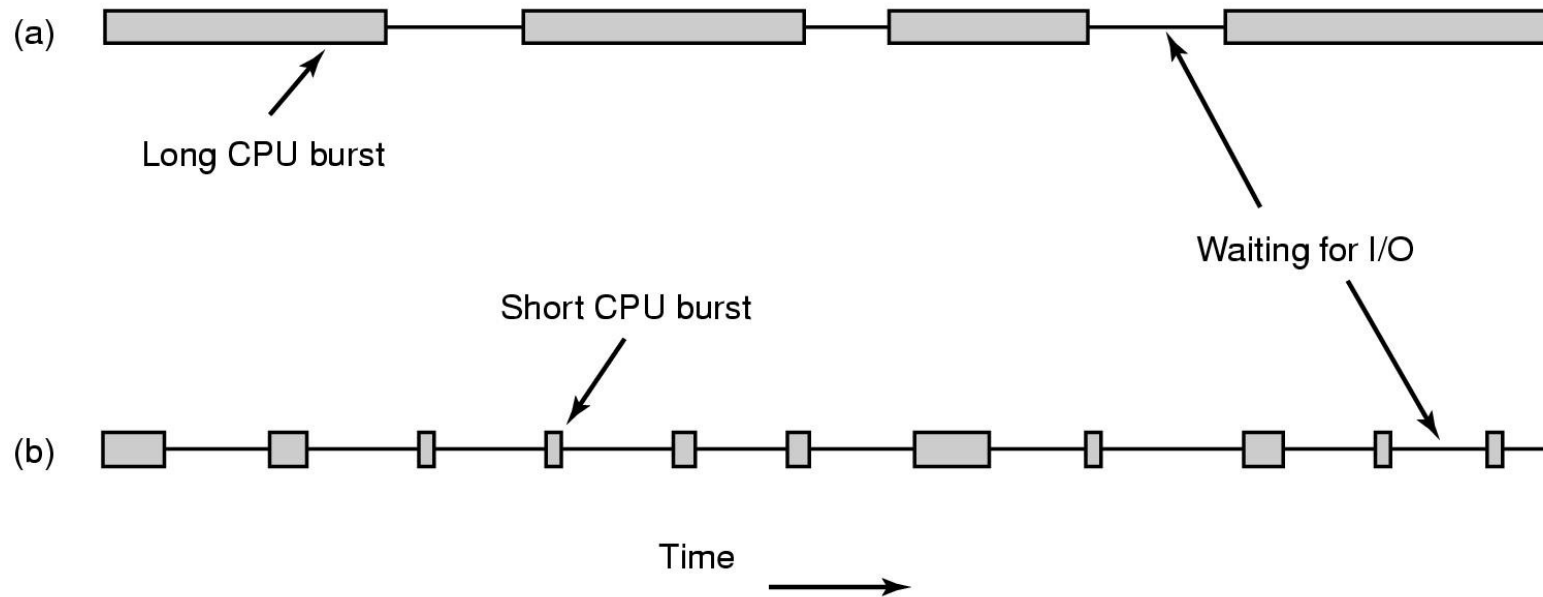
Ventajas:

- El bloqueo de un thread no bloquea todos los threads del proceso que lo contiene.

Desventajas:

- La creación/destrucción de threads es más cara en el kernel que en espacio de usuario (reciclar threads).
- El cambio de contexto de threads también es más caro

Planificación de procesos (1)



Planificar conjuntamente procesos CPU bound con procesos I/O bound

Planificar cuando: proceso creado, muerto o bloqueado, int. reloj o fin de E/S

CRITERIOS DE PLANIFICACIÓN DE LA CPU

- Tratar de utilizar el CPU al 100% todo el tiempo posible

Utilización de la CPU



- Cantidad de procesos que se ejecutan por unidad de tiempo

Tasa de Procesamiento



- El intervalo entre el instante que se ordena la ejecución de un proceso y el instante que se completa

Tiempo de Ejecución



- La suma de los periodos invertidos en esperar en la cola de procesos preparados

Tiempo de Espera



- Tiempo transcurrido entre que se envia una solicitud hasta que se produce la primera respuesta

Tiempo de Respuesta



Planificación de procesos (2)

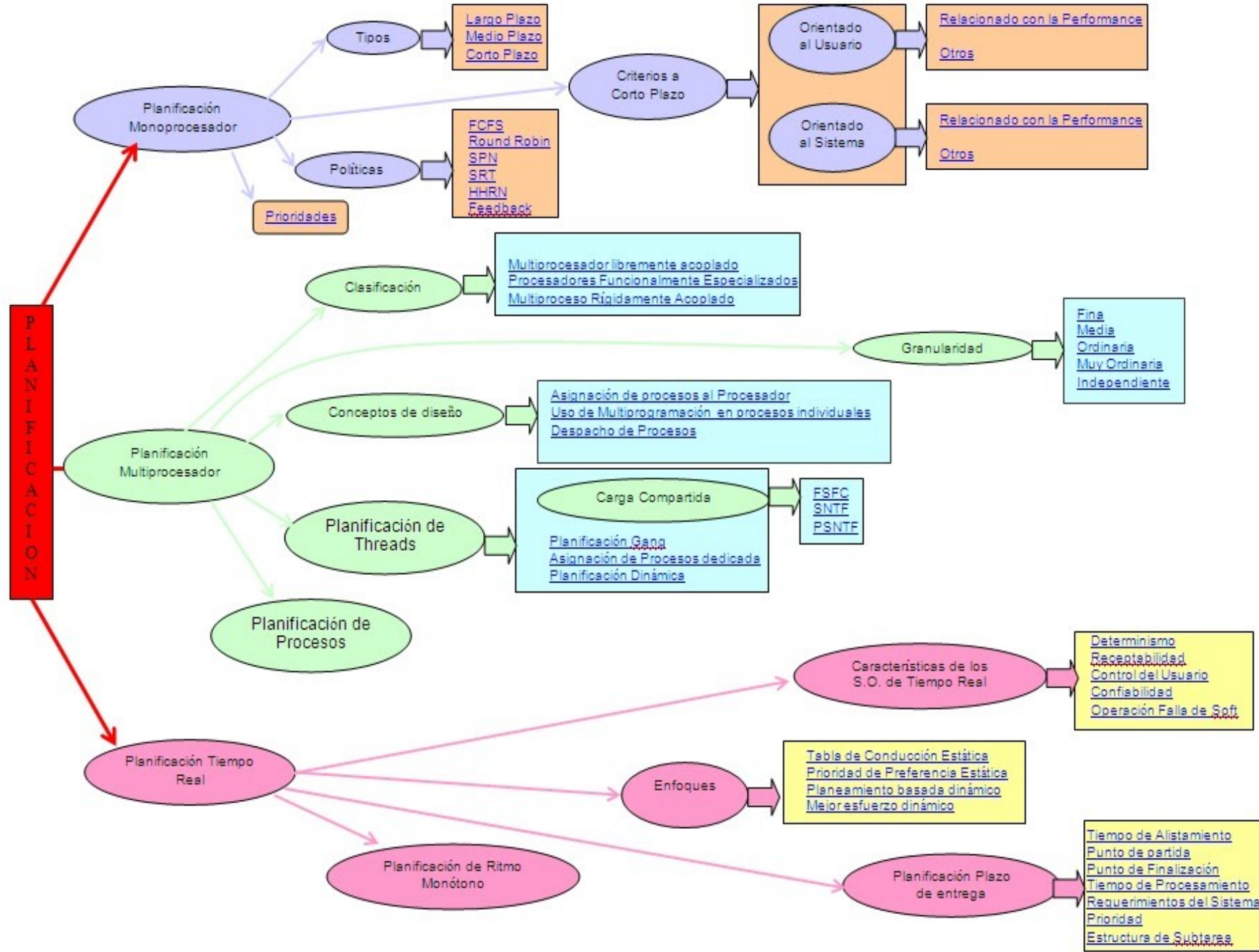
Criterios:

- ▶ Justicia (Fairness)
 - ▶ A trabajos parecidos, asignación parecida de CPU (todos)
- ▶ Eficiencia
 - ▶ % de CPU (y dispositivos de E/S) ocupada (todos)
- ▶ Tiempo de retorno (turnaround)
 - ▶ Tiempo medio desde que entra un trabajo hasta que sale (batch)
- ▶ Tiempo de respuesta (response time)
 - ▶ Tiempo entre que se da un comando y se obtiene la respuesta (interactive)
- ▶ Tiempo de espera
 - ▶ Tiempo en ejecutarse todos los trabajos delante suyo (batch)
- ▶ Rendimiento (throughput)
 - ▶ Número de trabajos completados por unidad de tiempo (batch)

Planificación de procesos (3)

Balances:

- ▶ (Batch). Ejecuto preferentemente trabajos pequeños
 - ▶ Alto rendimiento, pero mal turnaround para trabajos grandes.
- ▶ (Batch + interactivos) preferencia a los interactivos; los batch de noche
 - ▶ Maximizo el rendimiento
 - ▶ Para los interactivos buen tiempo de respuesta
 - ▶ Para los batch mal turnaround



Planificación de procesos (4)

Categorías de algoritmos de planificación:

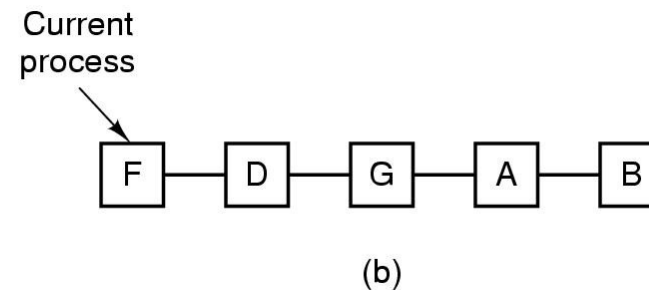
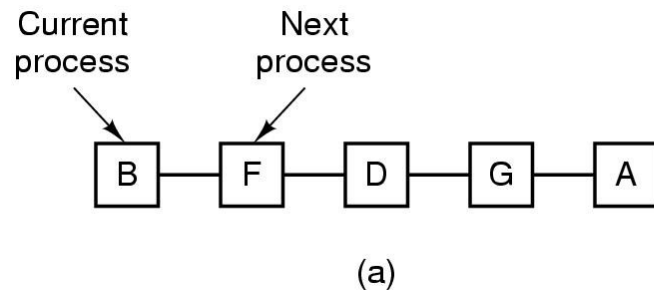
- ▶ Sistemas Batch
 - ▶ No expulsos o expulsos con un quantum grande
 - ▶ Reducen cambios de contexto y mejoran el rendimiento
 - ▶ FCFS, SJF, SRTN, por niveles
- ▶ Sistemas interactivos
 - ▶ Expulsos: evita la monopolización de la CPU
 - ▶ Round-Robin, prioridades, múltiples colas, SPN (envejecimiento)
- ▶ Sistemas de tiempo real
 - ▶ Monotónico en frecuencia
 - ▶ Deadline más próximo el siguiente

Planificación de procesos (10)

Sistemas Interactivos

Round-Robin Robo de ciclo

- ▶ Todos los procesos tienen la misma importancia.
- ▶ A cada proceso se le asigna un **quantum** (rodaja) de tiempo.



¿Tamaño del quantum?

- Si demasiado pequeño, poco aprovechamiento CPU
- Si demasiado grande, pobre tiempo de respuesta

Planificación de procesos (11)

Sistemas Interactivos

Por prioridades

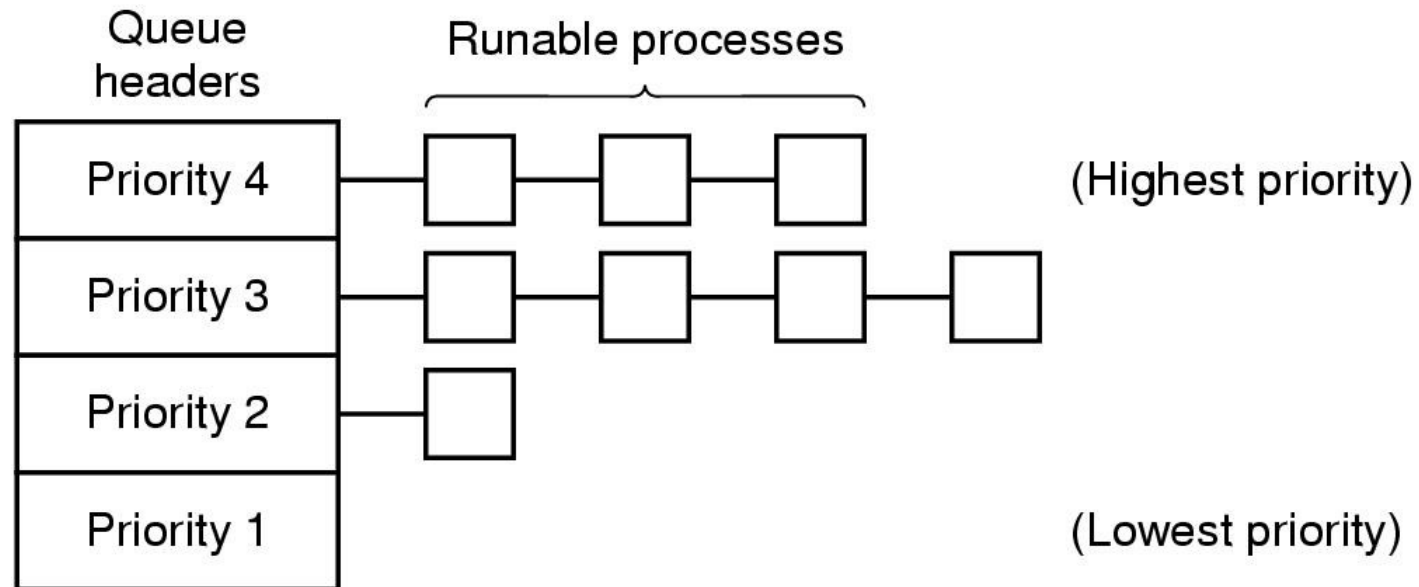
Interactivo (expulsor?)

- ▶ Cada proceso tiene asignada una prioridad
- ▶ Seleccionar el proceso más prioritario
- ▶ Posible inanición de los procesos menos prioritarios.
Solución:
 - ▶ Decrementar la prioridad del proceso ejecutándose cada tick
 - ▶ Round-robin entre procesos de la misma prioridad
- ▶ Asignación de prioridades: estática o dinámica
 - ▶ Estática: por pago, por clases de usuarios
 - ▶ Dinámica:
 - ▶ $1/f$, /* f = fracción consumida del último quantum */
- ▶ Clases de prioridad

Planificación de procesos (12)

Sistemas Interactivos

Por prioridades (clases de prioridad)



Planificación de procesos (13)

Sistemas Interactivos

Múltiples colas

- ▶ CTSS(*Compatible Time-Sharing System*): clases de prioridad:
 - ▶ 1ª clase: 1 quantum
 - ▶ 2ª clase: 2 quantum
 - ▶ 3ª clase: 4 quantum
 - ▶ Si un proceso agota su quantum, se pasa a la siguiente clase menor
 - ▶ Ejemplo: un proceso necesita 100 quanta.
 - ▶ 1, 2, 4, 8, 16, 32, 64
 - ▶ 7 intercambios
 - ▶ Cuando un proceso se convierte en interactivo: a la clase más alta
- ▶ XDS 940: 4 clases de prioridad:
 - ▶ Terminal, I/O, short quantum, long quantum

Planificación de procesos (6)

Sistemas Batch

Trabajo más corto el próximo (SJF) Shortest Job First.

☺ Ofrece siempre el mínimo tiempo medio de espera

<u>Trabajo</u>	<u>Tiempo necesario</u>
1	24
2	3
3	3



$$\text{Tiempo medio de espera} = (0+3+6) / 3 = 3$$

Planificación de procesos (5)

Sistemas Batch

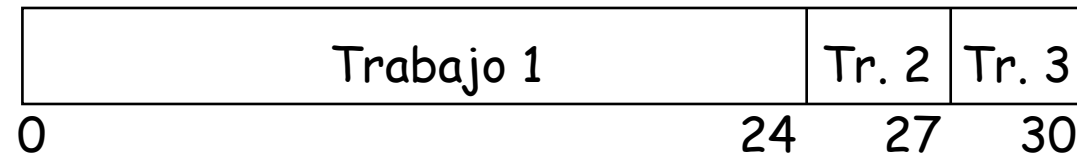
Primero en llegar, primero servido (FCFS)

First-Come First-Served.

☺ Es simple.

☹ Tiempo de espera variable. Raramente el mínimo.

<u>Trabajo</u>	<u>Tiempo necesario</u>
1	24
2	3
3	3



$$\text{Tiempo medio de espera} = (0+24+27) / 3 = 17$$

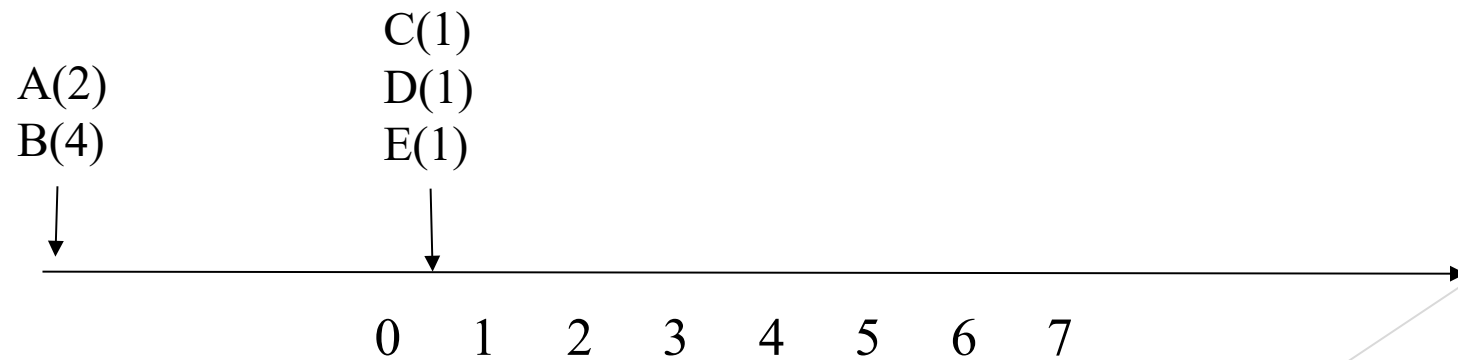
Planificación de procesos (8)

Sistemas Batch

Tiempo restante menor, el siguiente (SRTN)

Shortest Remaining Time Next.

- Variante expulsora del SJF.
- Cuando llega un trabajo nuevo, comparar su petición de tiempo con el tiempo que le queda al actual. Seleccionar el menor.
- Favorece a los trabajos nuevos



Planificación de procesos (14)

Sistemas Interactivos

Shortest Process Next (Envejecimiento)

Variante del SJF para interactivo, (No expulsor)

- ▶ SJF genera el mínimo tiempo medio de espera
- ▶ ¿Se puede aplicar SJF a sistemas interactivos?
- ▶ Algoritmo del **envejecimiento** (aging)
 - ▶ Estimación actual basada en comportamiento pasado
 - ▶ Primer comando: T_0
 - ▶ Segundo comando: T_1
 - ▶ Estimación del siguiente: $aT_0 + (1-a)T_1$
 - ▶ Si $a=1/2$, sumar el nuevo valor a la estimación actual y dividir por 2.

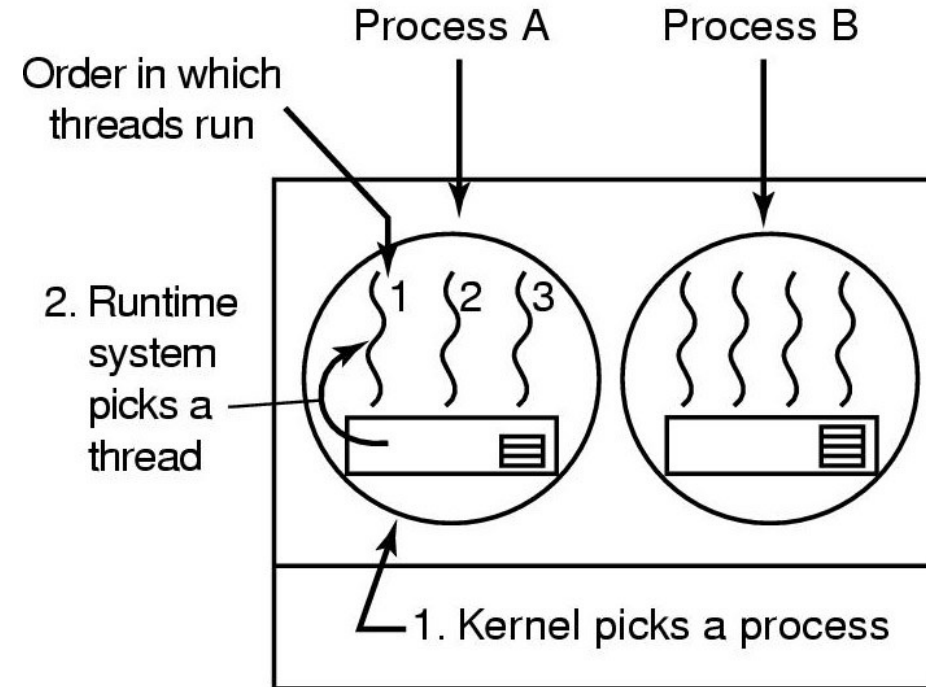
(Guaranteed Scheduling,
Lottery Scheduling, Fair-Share Scheduling)

Planificación a la Tasa de Respuesta más Alta HRN

- ▶ (HRN, highest-response-ratio-next)
- ▶ corrige algunas deficiencias de SJF,
 - ▶ el retraso excesivo de trabajos largos
 - ▶ el favoritismo excesivo para los trabajos cortos.
- ▶ no apropiativa
- ▶ la prioridad de cada proceso
 - ▶ no sólo se calcula en función del tiempo de servicio,
 - ▶ sino también del tiempo que ha esperado para ser atendido.
 - ▶ Cuando un trabajo obtiene el procesador, se ejecuta hasta terminar.

Planificación de procesos (17)

Planificación de threads



Possible: A1, A2, A3, A1, A2, A3

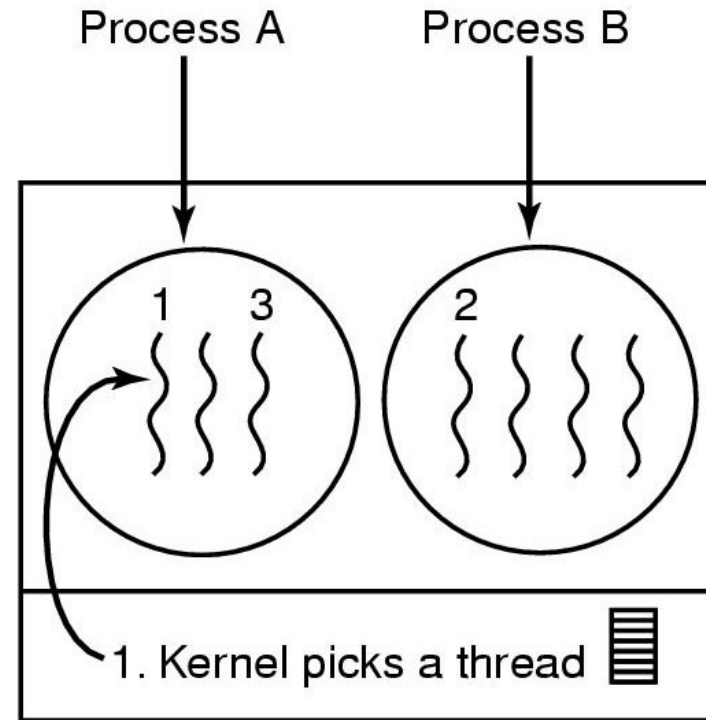
Not possible: A1, B1, A2, B2, A3, B3

Possible planificación de threads en espacio de usuario

- Quantum por proceso de 50-msec
- Cada threads ejecuta 5 msec/CPU burst

Planificación de procesos (18)

Planificación de



Possible: A1, A2, A3, A1, A2, A3

Also possible: A1, B1, A2, B2, A3, B3

Possible planificación de threads en espacio de kernel

- Quantum por proceso de 50-msec
- Cada threads ejecuta 5 msec/CPU burst

Comunicación entre procesos (1)

Los procesos necesitan comunicarse y sincronizarse.
Se presentan tres problemas:

- Cómo pasar información de un proceso a otro
 - Threads (espacio compartido)
 - Procesos (espacio no compartido)
- Exclusión mutua sobre regiones críticas
- Secuenciamiento cuando existen dependencias

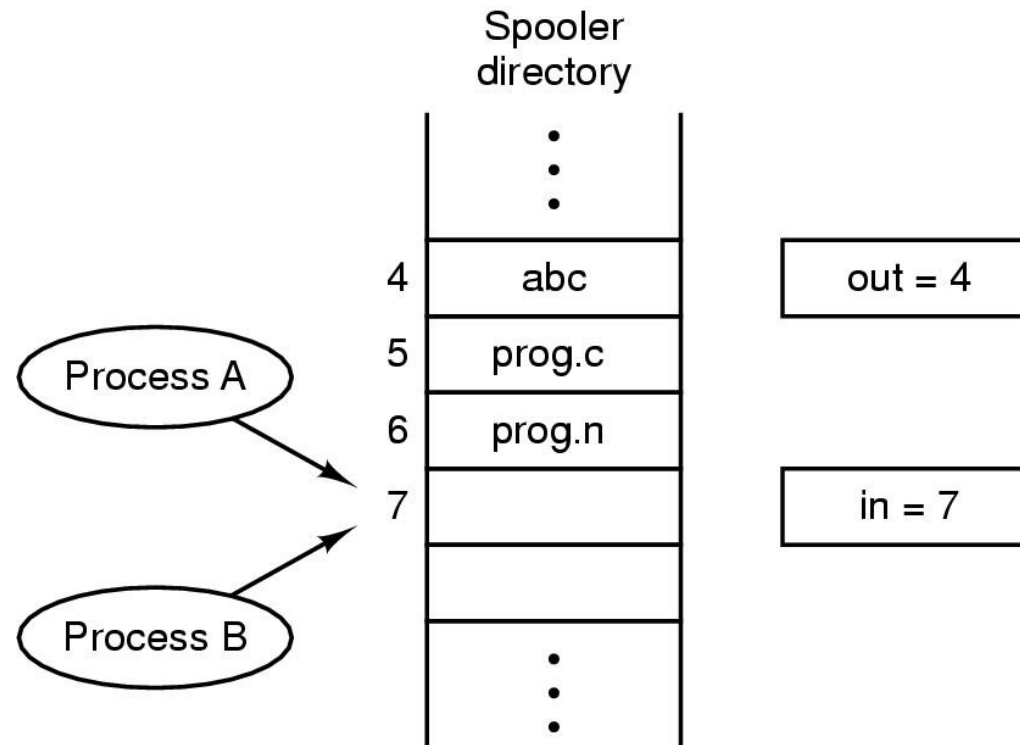
Problemas de la concurrencia

- ▶ Condiciones de carrera
- ▶ Aplazamiento indefinido
- ▶ Espera circular
- ▶ No apropiacion
- ▶ Espera Ocupada
- ▶ Exclusion mutua
- ▶ Ocupar y esperar un recurso

Comunicación entre procesos (2)

Condiciones de carrera

Condición de carrera: el resultado del cálculo depende de las velocidades relativas de los procesos



Comunicación entre procesos (3)

Condiciones de carrera

```
program Cuenta_Eventos;  
  Contador : INTEGER := 0;  
  
  process Observador;  
    repeat  
      Esperar_Evento;  
      Contador := Contador + 1;  
    forever;  
  end Observador;  
  
  process Reportero;  
    repeat  
      Imprimir (Contador);  
      Contador := 0;  
    forever;  
  end Reportero;  
  
begin  
  Observador;  
  Reportero;  
end Cuenta_Eventos;
```

Comunicación entre procesos (4)

Región crítica

Región crítica: exclusión mutua sobre el recurso compartido

```
process Observador;  
  repeat  
    Esperar_Evento;  
    Entrar_RC;  
    Contador := Contador + 1;  
    Salir_RC;  
  forever;  
end Observador;
```

```
process Reportero;  
  repeat  
    Entrar_RC;  
    Imprimir (Contador);  
    Contador := 0;  
    Salir_RC;  
  forever;  
end Reportero;
```

Comunicación entre procesos (5)

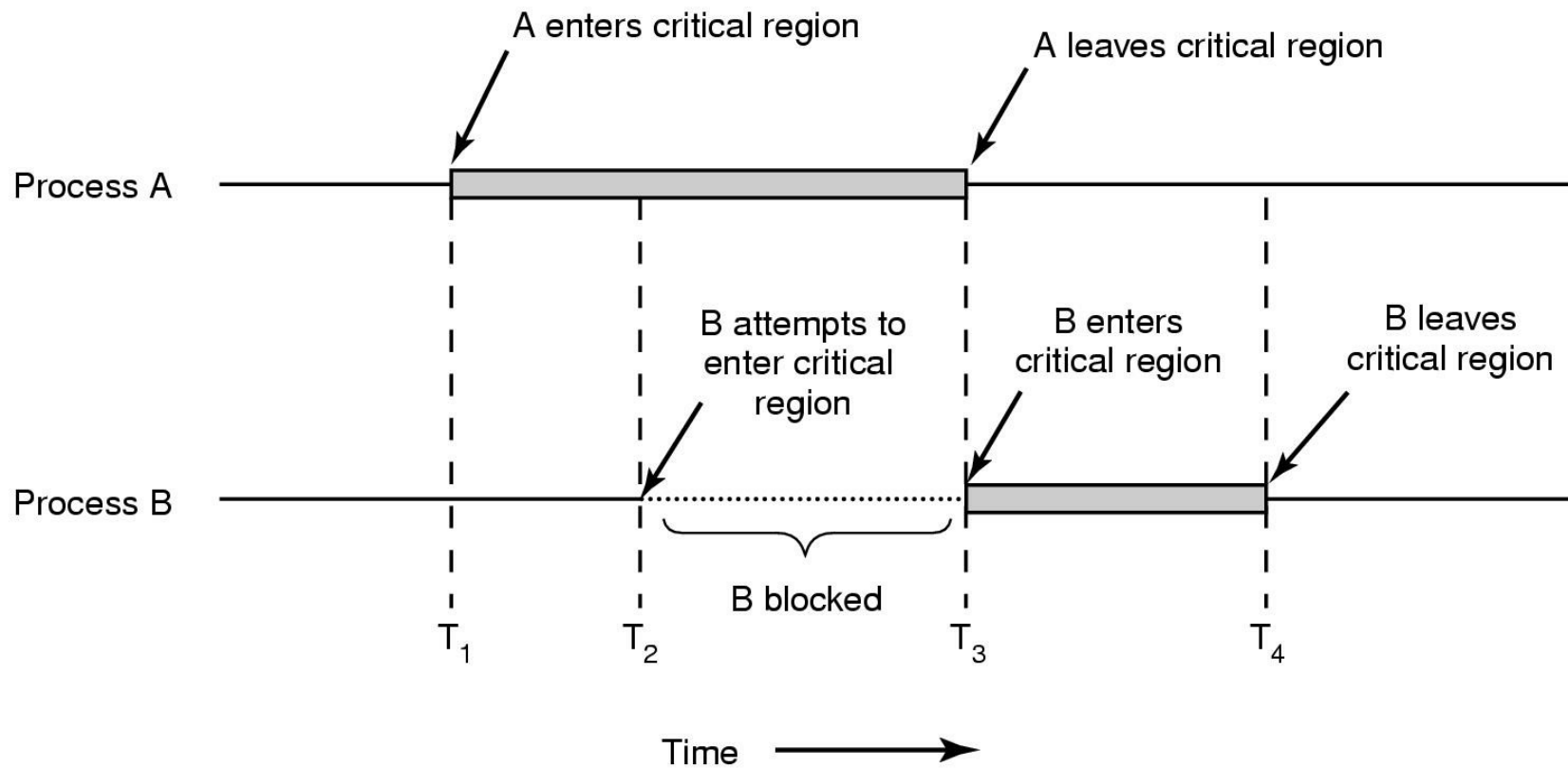
Región crítica

Región crítica: condiciones de la exclusión mutua

- Dos procesos no pueden estar al mismo tiempo dentro de la misma región crítica
- No se deben hacer suposiciones sobre el número o velocidad de las CPUs (velocidad relativa de los procesos)
- Ningún proceso fuera de una región crítica puede bloquear a otros procesos
- Antes o después, un proceso que quiera entrar a una RC lo hará
- La decisión de qué proceso entra a una región crítica se toma en un tiempo finito

Comunicación entre procesos (6)

Región crítica



Comunicación entre procesos (7)

Exclusión mutua con espera activa

Mecanismo: Inhibir interrupciones

1. entrar: inhibir; salir: permitir
2. dentro de la RC, un proceso no pide E/S

PERO:

- pueden perderse interrupciones
- se impide que otros procesos ajenos puedan continuar
- no vale si hay más de una CPU
- Es peligroso darle al usuario la facultad de inhibir interrupciones

Dentro del kernel, sí se inhiben interrupciones

Comunicación entre procesos (8)

Exclusión mutua con espera activa

Mecanismo: Variable cerrojo

variable = 0, RC libre; variable = 1, RC ocupada

protocolo de entrada

if variable = 0

then variable:=1; entrar en RC

else esperar hasta que variable = 0

protocolo de salida

variable:= 0

Mismo problema que el problema del Observador, Reportero.

Comunicación entre procesos (9)

Exclusión mutua con espera activa

Mecanismo: Test and Set Lock

TSL RX, LOCK

Lee el contenido de LOCK en RX y pone un valor distinto de 0 en LOCK

enter_region:

TSL REGISTER, LOCK

| copy lock to register and set lock to 1

CMP REGISTER, #0

| was lock zero?

JNE enter_region

| if it was non zero, lock was set, so loop

RET | return to caller; critical region entered

leave_region:

MOVE LOCK, #0

| store a 0 in lock

RET | return to caller

Comunicación entre procesos (11)

Semáforos

Package Semaforos is

```
type SEMAFOROS is private;
```

```
procedure Inicializar(S: SEMAFOROS;  
                      Valor : INTEGER);
```

```
procedure Bajar (S: SEMAFOROS);
```

```
procedure Subir (S: SEMAFOROS);
```

```
end Semaforos;
```

Comunicación entre procesos (12)

Semáforos

```
Program Cuenta_Eventos;  
  Contador : INTEGER;  
  S        : SEMAFOROS;  
  
  process Observador;  
    repeat  
      Esperar_Evento;  
      Bajar (S);  
      Contador := Contador + 1;  
      Subir (S);  
    forever;  
  end Observador;  
  
  process Reportero;  
    repeat  
      Bajar (S);  
      Imprimir (Contador);  
      Contador := 0;  
      Subir (S);  
    forever;  
  end Reportero;
```

```
begin  
  Inicializar (S, 1);  
  Observador;  
  Reportero;  
end Cuenta_Eventos;
```

Comunicación entre procesos (13)

Semáforos

```
type SEMAFOROS is private;
procedure Inicializar(S      : SEMAFOROS;
                      Valor : INTEGER);
procedure Bajar (S : SEMAFOROS);
procedure Subir (S : SEMAFOROS);
private -- Inaccesible al usuario
  type SEMAFOROS is
    record
      Contador : INTEGER;
      Cola     : COLA_PROCESOS;
    end;
  --
  procedure Inicializar (S      : SEMAFOROS;
                        Valor : INTEGER) is
  begin
    S.Contador := Valor;
  end Inicializar;
```

Comunicación entre procesos (14)

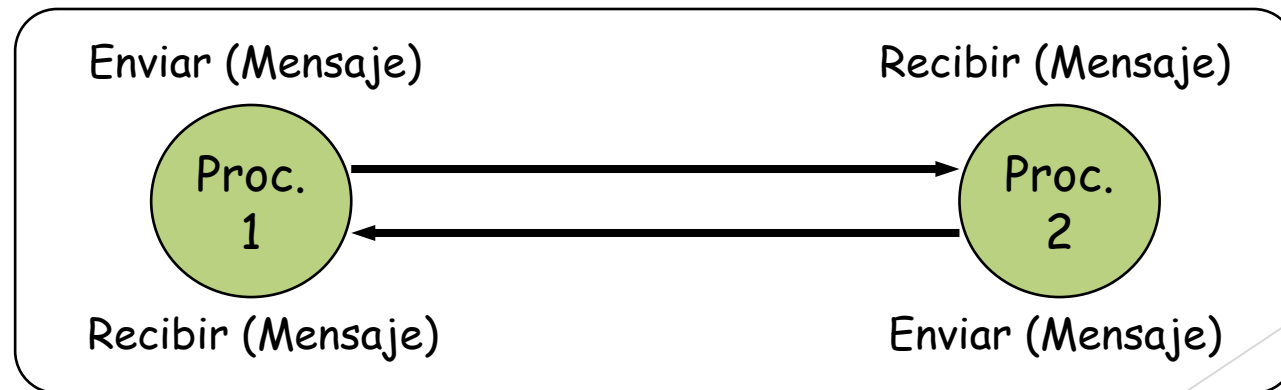
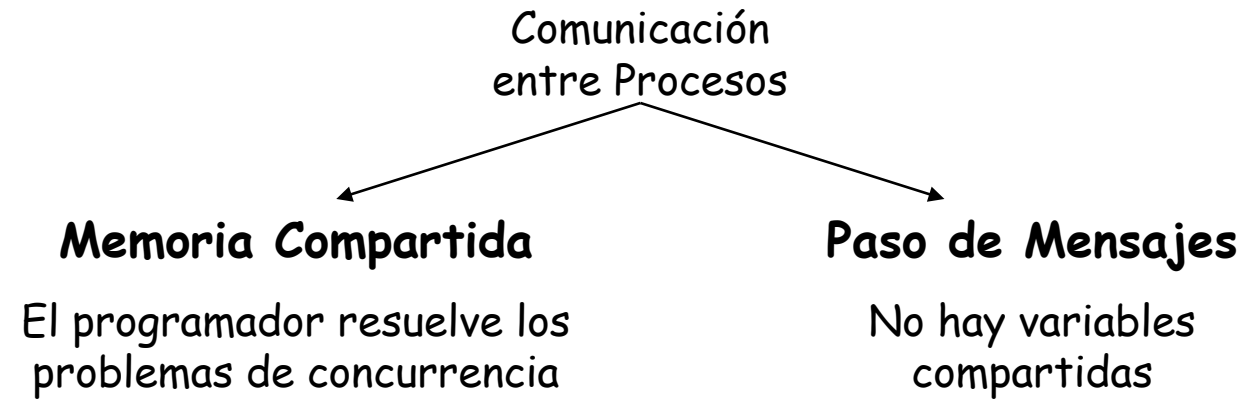
Semáforos

```
procedure Bajar (S : SEMAFOROS) is
begin
    if S.Contador < 1 then
        Encolar (Este_Proceso, S.Cola);
        Suspendir; -- Implica llamada al Planificador
    else
        S.Contador := S.Contador - 1;
    endif;
end Bajar;

procedure Subir (S : SEMAFOROS) is
    Proceso : ID_PROCESO;
begin
    if s.Cola.Primerio /= 0 then -- Si algun proc. Esperando
        Desencolar (Proceso, S.Cola);
        Preparar (Proceso); -- Llamada al Planificador
    else
        S.Contador := S.Contador + 1;
    endif;
end Subir;
```

Comunicación entre procesos (15)

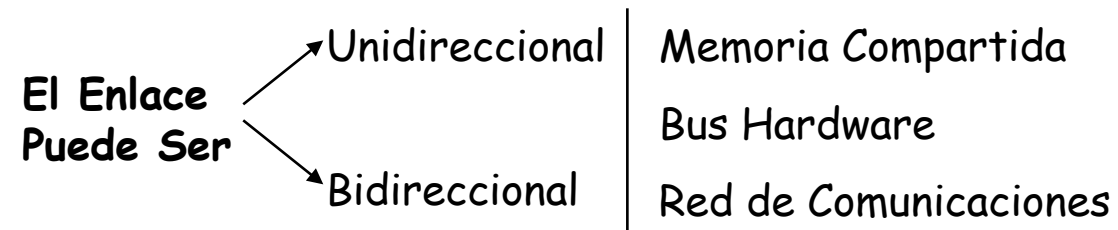
Paso de mensajes



Comunicación entre procesos (16)

Paso de mensajes

ALGUNAS CUESTIONES



Modelos Lógicos	Comunicación DIRECTA o INDIRECTA
	Comunicación SIMÉTRICA O ASIMÉTRICA

Capacidad del Buzón

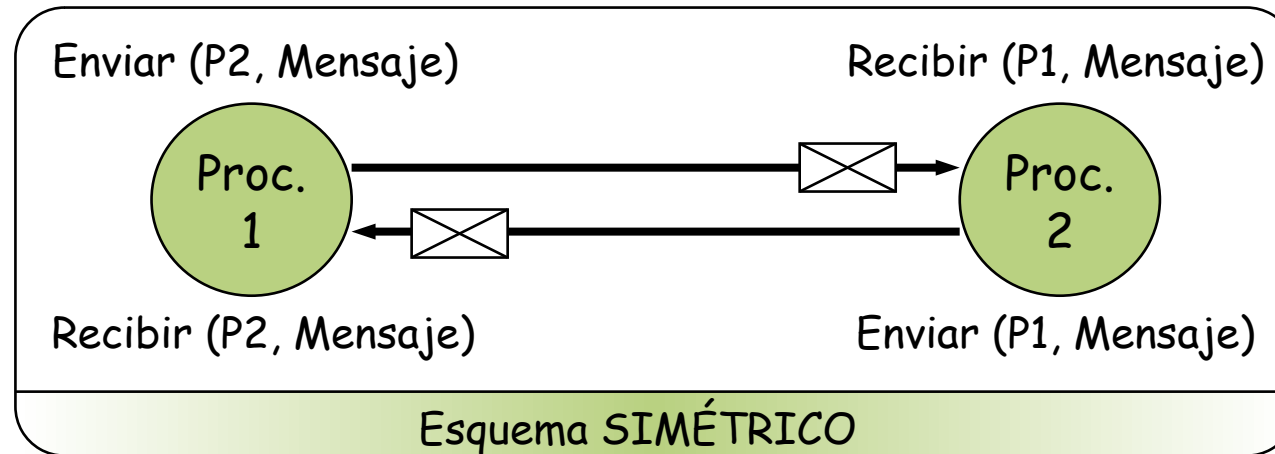
Tamaño del Mensaje	Fijo
	Variable

Comunicación entre procesos (17)

Paso de mensajes

Modelos comunicación. Comunicación directa

- Cada proceso tiene su propio buzón.
- Cada proceso implicado debe indicar explícitamente el nombre del receptor o emisor.



Observador

```
repeat
  . . .
  Esperar_Evento
  . . .
  Enviar (Reportero, Mensaje)
forever
```

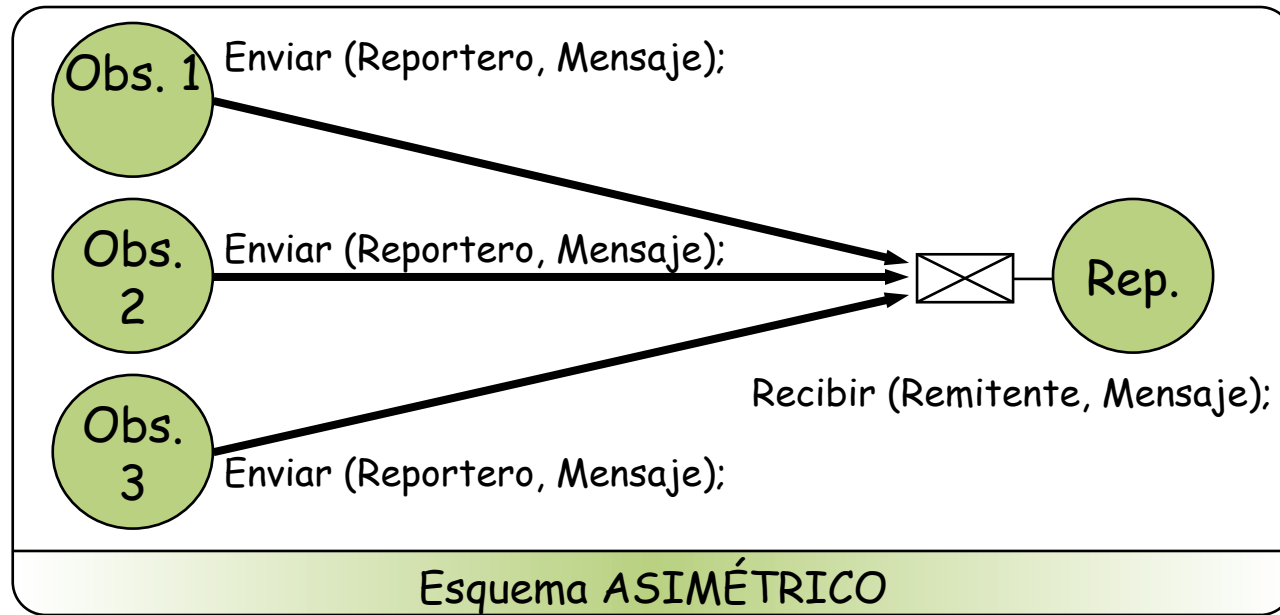
Reportero

```
repeat
  . . .
  Recibir (Observador, Mensaje)
  . . .
  Imprimir_Evento
forever
```

Comunicación entre procesos (18)

Paso de mensajes

Modelos comunicación. Comunicación directa



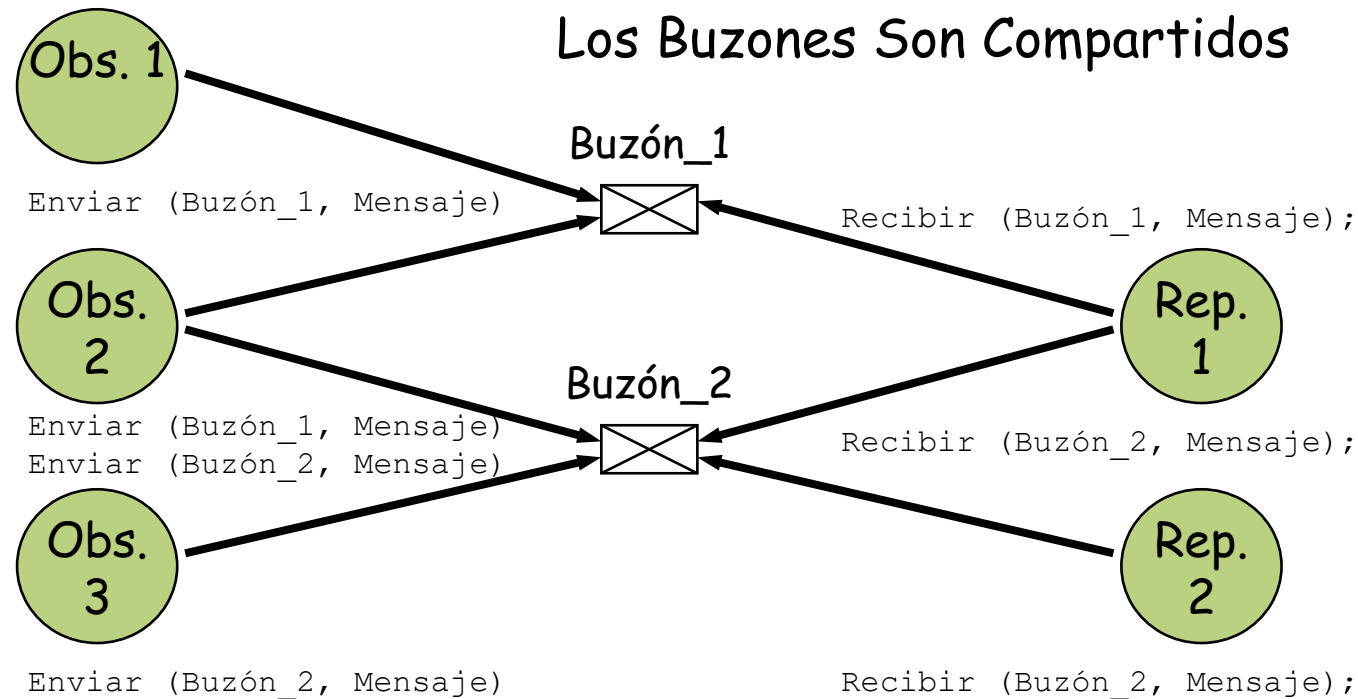
Pega de la
Comunicación
Directa

Si cambia el nombre de un proceso,
hay que revisar todas las referencias a él

Comunicación entre procesos (19)

Paso de mensajes

Modelos comunicación. Comunicación indirecta



Los mensajes se envían y reciben a/de buzones, no a procesos.

Comunicación entre procesos (20)

Paso de mensajes

Tamaño del buzón

- Capacidad Limitada (n mensajes)

Si hay espacio $\Rightarrow$ El emisor continúa la ejecución después del envío

Si está lleno $\Rightarrow$ - El emisor queda bloqueado hasta que haya espacio en el buzón para dejar un mensaje.

o

- Se devuelve un *status* = lleno

- Capacidad Ilimitada
El emisor nunca se bloquea en el envío de mensajes

- Capacidad Nula (rendez-vous) Síncrono

El emisor queda bloqueado hasta que el receptor está listo para recibir el mensaje.

Asíncrono